Artificial Intelligence Technology and Application

Python
Lab Guide

Student Version

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD.

Contents

1 Regular Expression Experiment 1
1.1 Introduction 1
1.1.1 Regular Expressions 1
2 Generators, Iterators, and Decorators 4
2.1 Procedure 4
2.1.1 Iterator 4
2.1.2 Generators 4
2.1.3 Decorators 5

1 Regular Expression Experiment

1.1 Introduction

1.1.1 Regular Expressions

Step 1 re.match function

The re.match function attempts to match a pattern from the start position of the string. If the match is not successful, match() returns none.

Function syntax:

```
```

Example:

```
import re
# Match at the start position.

# Not match at the start position.
```

Output:

```
(0, 3)
None
```

Step 2 re.search method

The re.search method scans the entire string and returns the first successful match.

Function syntax:

```
```

Example:

```
import re
```

Output:

```
searchObj.group() :   Cats are smarter than dogs
searchObj.group(1) :   Cats
searchObj.group(2) :   smarter
```

Step 3 Retrieval and replacement

The re module of Python provides re.sub function to replace the matching items in the string.

Syntax:

```

```

Example:

```
import re
phone = "2019-0101-000 # This is a phone number."
# Delete the Python comment in the string.


# Delete the hyphens from the phone number.
```

Output:

```
Phone number:   2019-0101-000
Phone number: 20190101000
```

Step 4 re.compile function

The compile function is used to compile regular expressions and generate a regular expression object (pattern) for the match() and search() functions.

The syntax format is as follows:

```

```

Example:

```
import re
# At least one digit is matched.

# The header is not found.
```

Output:

```
None
<_sre.SRE_Match object; span=(3, 5), match='12'>
12
```

Step 5 re.split()

The split method splits a string based on the matched substring and returns a list. The usage of the method is as follows:

```
```

Example:

```
import re
s = re.split('\W+', 'www.huawei.com')
print(s)
```

Output:

```
['www', 'huawei', 'com']
```

2 Generators, Iterators, and Decorators

2.1 Procedure

2.1.1 Iterator

Step 1 Determine an iterable object.

```
# Use the isinstance() method to determine whether an object can be iterated.
# Iterable object.
```

Output:

```
True
True
False
```

Step 2 Obtain the iterator.

The iter() function is used to obtain the iterators of the iterable objects. You can use the next() function continuously for the obtained iterator to obtain the next data record.

```
l = [1, 2, 3, 4, 5]
```

Output:

```
False
True
```

2.1.2 Generators

Step 1 Create a generator through list derivation.

```
```

Output:

```
<class 'generator'>
```

Step 2 Create a generator through a yield keyword.

Use the yield keyword to create a generator to generate the Fibonacci sequence.

```
```

Output:

```
value:0
value:1
value:1
value:2
value:3
Generator return value: done
```

Step 3 Use the send method.

In addition to the next method, the send method can be used to wake up the generator. Compared with the next method, the send method can also transfer data to the breakpoint when waking up the generator.

```
>>>0
f.send('haha')
>>>haha
>>>1
next(f)
>>>None
>>>2
```

2.1.3 Decorators

Step 1 Construct a decorator.

Create a simple decorator that calculates the runtime of a function.

```
import time
```

```

```

Step 2 Use the decoration function.

```
@func
def test():
```

Output:

```
Time consumed: 2.000276803970337 seconds
```